

Probabilistic forecasting of renewable energy and electricity demand using Graph-based Denoising Diffusion Probabilistic Model

Amir Miraki ^a,*, Pekka Parviainen ^b, Reza Arghandeh ^a

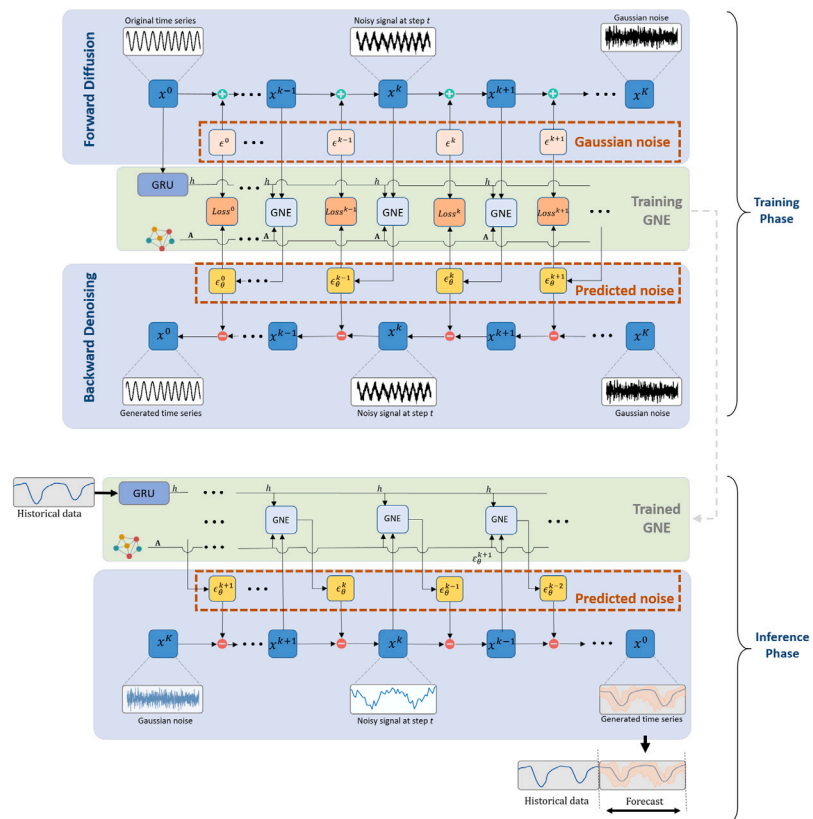
^a Department of Computer Science, Electrical Engineering and Mathematical Sciences, Western Norway University of Applied Sciences, Norway

^b Department of Informatics, University of Bergen, Norway

HIGHLIGHTS

- **Novel Forecasting Method:** Proposes the Graph-based Denoising Diffusion Probabilistic Model (G-DDPM) for multivariate time series forecasting, explicitly modeling interdependencies between variables using graph techniques.
- **Enhanced Feature Extraction:** Incorporates a parallel feature extraction module to capture both temporal dynamics and spatial correlations, significantly improving forecasting accuracy.
- **Probabilistic Forecasting Performance:** Demonstrates state-of-the-art performance in probabilistic forecasting tasks, validated through extensive evaluation on real-world renewable energy and electricity demand datasets.
- **Uncertainty Quantification:** Provides reliable probabilistic forecasts that effectively quantify uncertainties, crucial for decision-making and resource allocation in the energy sector.
- **Comprehensive Evaluation:** Extensively validated on two real-world datasets, showcasing its applicability and robustness in handling diverse variables originating from renewable energy sources and electricity demand patterns.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Multivariate time series

ABSTRACT

Renewable energy production and the balance between production and demand have become increasingly crucial in modern power systems, necessitating accurate forecasting. Traditional deterministic methods fail

* Corresponding author.
E-mail address: amir@hvl.no (A. Miraki).

<https://doi.org/10.1016/j.egyai.2024.100459>

Received 7 August 2024; Received in revised form 28 October 2024; Accepted 5 December 2024

Available online 12 December 2024

2666-5468/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Graph neural network
 Denoising diffusion probabilistic models
 Forecasting

to capture the inherent uncertainties associated with intermittent renewable sources and fluctuating demand patterns. This paper proposes a novel denoising diffusion method for multivariate time series probabilistic forecasting that explicitly models the interdependencies between variables through graph modeling. Our framework employs a parallel feature extraction module that simultaneously captures temporal dynamics and spatial correlations, enabling improved forecasting accuracy. Through extensive evaluation on two real-world datasets focused on renewable energy and electricity demand, we demonstrate that our approach achieves state-of-the-art performance in probabilistic energy time series forecasting tasks. By explicitly modeling variable interdependencies and incorporating temporal information, our method provides reliable probabilistic forecasts, crucial for effective decision-making and resource allocation in the energy sector. Extensive experiments validate that our proposed method reduces the Continuous Ranked Probability Score (CRPS) by 2.1%–70.9%, Mean Absolute Error (MAE) by 4.4%–52.2%, and Root Mean Squared Error (RMSE) by 7.9%–53.4% over existing methods on two real-world datasets.

1. Introduction

Accurate forecasting of electricity demand and supply has become increasingly crucial in modern power systems, especially with the rapid integration of renewable energy sources. As the world transitions towards a more sustainable energy future, the proliferation of intermittent renewable resources such as solar and wind power has introduced significant variability into the electricity grid [1,2]. Precise forecasting of electricity demand patterns and renewable generation is essential for efficient grid operation, resource allocation, and ensuring a reliable and cost-effective power supply [3].

Renewable energy sources like wind and solar are inherently intermittent, introducing intrinsic uncertainty into the power grid. Additionally, user behavior and electricity consumption patterns exhibit inherent uncertainties. Most traditional methods for energy time series forecasting are deterministic, providing single-point predictions [4–9]. However, deterministic forecasts are insufficient for effective decision-making and practical implementation, as they fail to capture the underlying uncertainties.

Early statistical approaches, such as additive quantile regression with boosting [10], laid the foundation for estimating probability distributions of future demand. Wavelet methods, exemplified by Wang et al.'s combined forecasting approach for wind speed prediction [11], further enhanced uncertainty quantification. The field then saw the application of machine learning techniques, with Gaussian Process Regression models employed for wind power forecasting [12] and Quantile Regression Forests used for solar power prediction [13]. Ensemble methods, like Hong et al.'s quantile regression averaging [14], improved forecast reliability by combining multiple models. Later, the field witnessed the emergence of more sophisticated approaches, including Graph Neural Networks [15], which model intricate spatial-temporal relationships in power systems. This paper [16] provides a comprehensive review of probabilistic energy forecasting.

Recently, generative AI methods, particularly denoising diffusion probabilistic models (DDPMs), have shown remarkable success in computer vision [17,18] and natural language processing tasks [19]. DDPMs are a class of generative models that learn a parameterized stochastic process through variational inference. The transitions of this process are learned to reverse a diffusion trajectory, which progressively corrupts the data by adding noise in the opposite direction of sampling until the original signal is obscured. These models offer several attractive properties, including comprehensive coverage of the data distribution, a stable and consistent training objective, and scalability to handle large-scale datasets [20]. DDPMs generate new samples by iteratively removing noise from an initially corrupted signal, following the learned reverse diffusion trajectory. The training objective of these models is formulated as a reweighted variational lower bound, which aims to optimize the model's ability to denoise the data and recover the original signal from the noisy input [21]. By learning the reverse diffusion trajectory, DDPMs can capture intricate data distributions and generate high-fidelity samples, making them particularly useful for

tasks such as energy time series forecasting, where accurately modeling complex distributions is crucial [22].

Researchers have explored the application of DDPMs, like TimeGrad [23], for probabilistic time series forecasting. However, these methods often struggle to capture the interdependencies between variables, which is crucial for accurate forecasting in energy systems. To bridge this gap, some recent works, such as GCRDD [24] and Spec-STG [25], have leveraged the power of graph neural networks to model the interdependencies between variables. While existing approaches have shown promise, they still lack the desired accuracy, especially in the energy domain where we have to deal with different variables originating from diverse sources.

In this paper, we propose a novel denoising diffusion method for multivariate time series probabilistic forecasting that explicitly considers the interdependencies between variables through graph modeling. Our method employs a parallel feature extraction module that captures both temporal and spatial information, enabling better forecasting performance. In summary, our main contributions are as follows:

- We introduce a novel framework called **Graph-based Denoising Diffusion Probabilistic Model (G-DDPM)** for probabilistic forecasting of multivariate time series data. This method leverages both temporal features and interdependencies between variables. Our proposed approach incorporates a parallel feature extraction module that simultaneously captures temporal dynamics and spatial correlations, enabling improved forecasting accuracy.
- Through extensive evaluation on two real-world datasets, one focused on renewable energy and the other on electricity demand, we demonstrate that our framework achieves state-of-the-art performance in probabilistic energy time series forecasting tasks.

The proposed framework offers a comprehensive solution for probabilistic forecasting in the energy domain, addressing the challenges posed by the inherent uncertainties associated with renewable energy sources and fluctuating electricity demand patterns. By accurately quantifying these uncertainties and providing well-calibrated probability distributions, our method empowers energy providers, grid operators, and policymakers to make informed decisions, optimize operations, and facilitate the seamless integration of renewable energy sources into the power grid.

2. Methodology

The objective of G-DDPM is to predict a future multivariate time series given a past context window of data. We define the data as $\mathbf{X} \in \mathbb{R}^{n \times d}$, where n is the number of variables, and d represents the number of time steps. For clarity, please note that in this paper, $\mathbf{X}$ denotes a matrix of data, while X denotes a column vector, representing the values of all variables at a given time step. Also note that we consider the original data as $\mathbf{X}^0 = \mathbf{X}$, where the superscript 0 indicates that the data is at the initial diffusion step.

In the context of multivariate time series forecasting, given observed values of the previous c time steps $X_{t-c}, \dots, X_{t-1}$, the task is

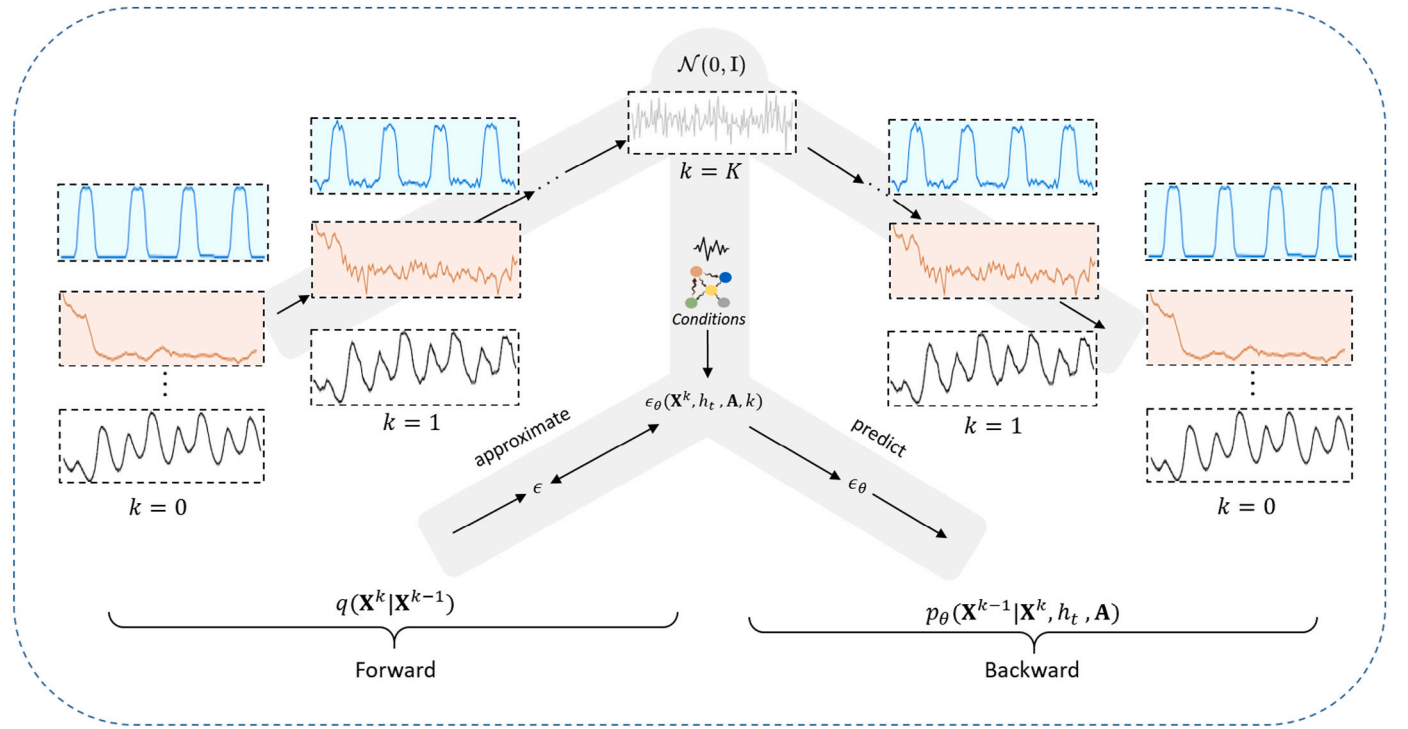


Fig. 1. Overview of denoising diffusion model for time series data. The process involves two key phases: a forward diffusion process and a backward diffusion process. In the forward process, the model starts with the original data sample and iteratively adds Gaussian noise to it over multiple timesteps until it becomes pure Gaussian noise. This forward diffusion process gradually destroys the data sample by adding more and more noise. During backward diffusion, the model learns to reverse this forward process by predicting the noise that was added at each step, allowing it to denoise the pure noise distribution back to the original data distribution.

to predict the variable values for the next f timestamps, denoted by $\hat{X}_t, \hat{X}_{t+1}, \dots, \hat{X}_{t+f-1}$. At a high level, these values can be inferred by the forecasting model F with parameters Φ , as represented by the following equation:

$$\hat{X}_t, \hat{X}_{t+1}, \dots, \hat{X}_{t+f-1} = F(X_{t-c}, \dots, X_{t-1}; \Phi),$$

In this study, we employ denoising diffusion probabilistic models to construct an effective forecasting model F .

Denoising diffusion models are a class of generative models that operate by gradually removing noise from a signal. Our proposed method builds on this concept and comprises two main components: the Forward Process and the Backward Process, centered around the idea of adding noise to data and subsequently denoising it. A general overview of the time series denoising diffusion model is illustrated in Fig. 1. The core idea is to learn the backward process of a forward diffusion process that adds noise to the data in a controlled manner. In the Forward Process, we start with the original data distribution and iteratively introduce Gaussian noise, transforming the complex data distribution into a simpler, tractable distribution, such as a Gaussian distribution, until the data becomes completely corrupted and indistinguishable from pure noise. The Backward Process employs a denoising network, which is specifically designed to handle and eliminate noise, effectively reconstructing the original signal. This reverse diffusion process, or denoising process, aims to learn the reverse of the forward process by starting with pure noise and iteratively removing the added noise to recover the original data distribution.

Fig. 2 illustrates the general pipeline of the proposed approach. It consists of two main phases: training and inference. Within the training phase, we have both forward and backward processes. The figure shows that this process incorporates several important components, including Gated Recurrent Units (GRU) and a Graph-based Noise Estimator (GNE). GRU is employed to extract hidden states, yielding an enhanced representation of the data. These representations enable the model to achieve better performance by providing a more informative and compact encoding of the input data. The GNE serves as our noise estimator

network. After training the model as shown in Fig. 2, we use the trained model for inference. With this high-level overview established, we can now proceed to examine the specifics of our proposed method in greater detail, exploring each component and its role in the overall system.

As previously mentioned, the data $\mathbf{X} \in \mathbb{R}^{n \times d}$, where n denotes the number of variables and d represents the number of time steps. Each of the temporal variables of multivariate time series can be treated as nodes or vertices in a graph. The edge connections (interdependencies) between nodes can be represented by a graph adjacency matrix $\mathbf{A}$.

Consider a given data $\mathbf{X}^0$ with diffusion steps k on the superscript, which follows the true data distribution $q(\mathbf{X}^0)$ over some data input. Let $p_\theta(\mathbf{X}^0)$ denote the probability density function that aims to approximate $q(\mathbf{X}^0)$ and allows for easy sampling. The generative process is unknown and needs to be learned. The approximate posterior $q(\mathbf{X}^{1:K} | \mathbf{X}^0)$ represents the diffusion process:

$$q(\mathbf{X}^{1:K} | \mathbf{X}^0) = \prod_{k=1}^K q(\mathbf{X}^k | \mathbf{X}^{k-1}) \quad (1)$$

and follows Markov chain. This process is generated by gradually adding Gaussian noise to $\mathbf{X}^{k-1}$ to obtain $\mathbf{X}^k$, and it is known as the forward diffusion process.

Fig. 2 depicts the overall pipeline of the proposed Graph-based Denoising Diffusion Probabilistic Model (G-DDPM). Each component of this pipeline will be explained in detail in the following sections.

2.1. Forward process

The forward process is as follows:

$$q(\mathbf{X}^k | \mathbf{X}^{k-1}) \sim \mathcal{N}(\mathbf{X}^k; \sqrt{1 - \beta_k} \mathbf{X}^{k-1}, \beta_k \mathbf{I}), \quad (2)$$

and here we apply various variances $\beta_1, \dots, \beta_k$ with $\beta_k \in (0, 1)$. At each step k , Gaussian noise is added to the data from the previous time step, following a predefined variance schedule: $\beta_1 < \beta_2 < \dots < \beta_k < 1$. The variance schedule determines how much noise is introduced at each

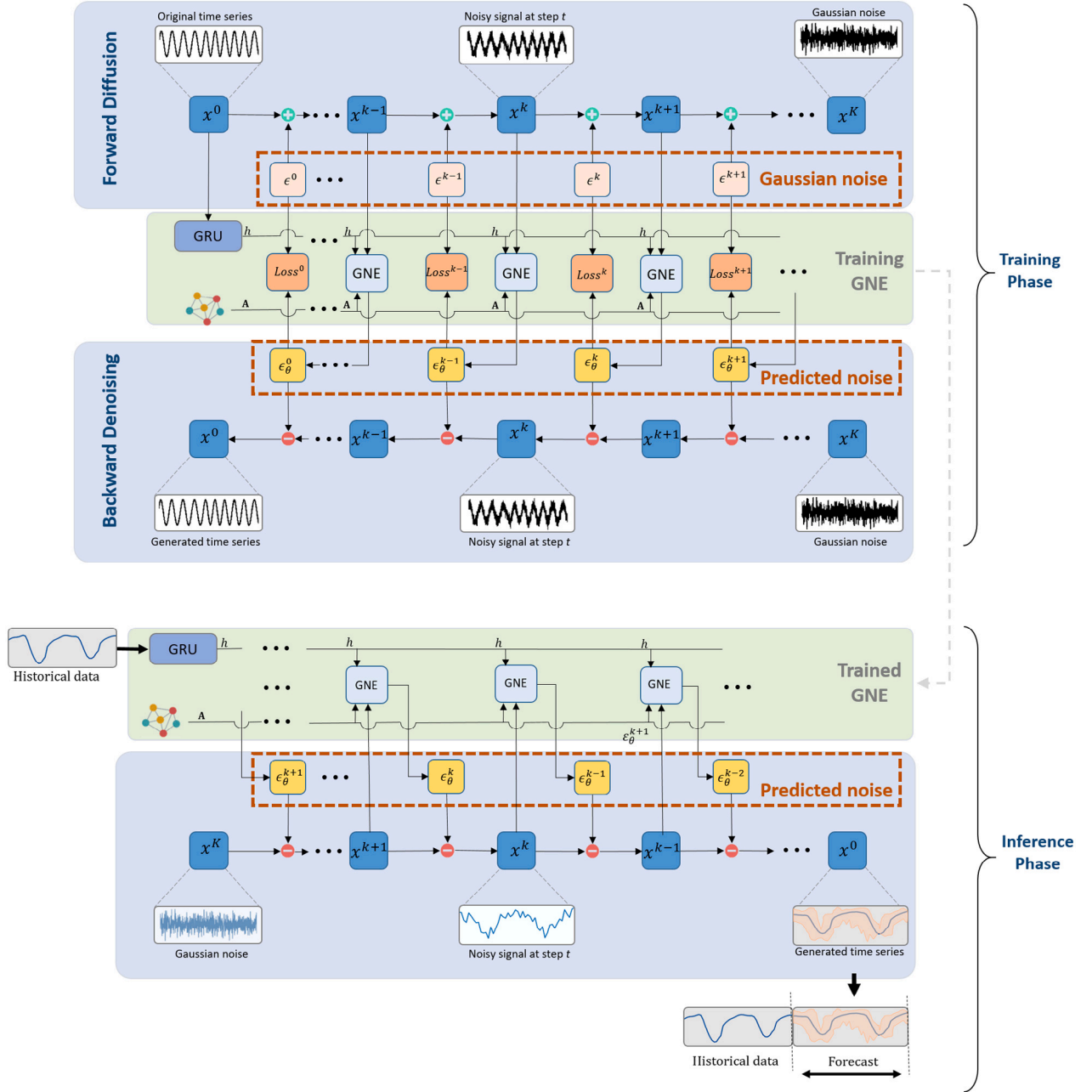


Fig. 2. General pipeline of the proposed Graph-based Denoising Diffusion Probabilistic Model (G-DDPM). During training, the model learns to reverse a forward diffusion process that adds Gaussian noise to data samples until they become pure noise. It does this by learning to predict the noise added at each step using the Graph-based Noise Estimation (GNE) block. During inference, the trained model generates new samples from the learned distribution by applying the learned denoising function iteratively to pure noise while considering the conditions, removing noise step-by-step until a clean sample is generated.

step. Smaller β values correspond to less noise, while larger values introduce more noise.

2.2. Backward process

The backward process, denoted by the joint distribution $p_\theta(\mathbf{X}^{0:K})$, operates as a Markov chain with a Gaussian transition learned from $p(\mathbf{X}^K) \sim \mathcal{N}(\mathbf{X}^K; 0, \mathbf{I})$ [17]. The subsequent transition is represented as:

$$p_\theta(\mathbf{X}^{0:K}) = p(\mathbf{X}^K) \prod_{k=1}^K p_\theta(\mathbf{X}^{k-1} | \mathbf{X}^k). \quad (3)$$

This expression captures the sequential evolution of the backward process, where each step is conditioned on the previous state, thereby

forming a Markov chain. Eq. (3) is defined by the parameterization as follows:

$$p_\theta(\mathbf{X}^{k-1} | \mathbf{X}^k) \sim \mathcal{N}(\mathbf{X}^{k-1}; \mu_\theta(\mathbf{X}^k, k), \Sigma_\theta(\mathbf{X}^k, k) \mathbf{I}), \quad (4)$$

where θ represents the learning parameters. The purpose of $p_\theta(\mathbf{X}^{k-1} | \mathbf{X}^k)$ is to extract the Gaussian noise added to $\mathbf{X}^k$. The objective of the learned parameters θ is to align with the data distribution $q(\mathbf{X}^0)$ by minimizing the negative log-likelihood.

Let's define $\alpha_k := 1 - \beta_k$ and the cumulative product of α_k as $\bar{\alpha}_k := \prod_{i=1}^k \alpha_i$. From Eq. (3), it's evident [17]:

$$q(\mathbf{X}^{k-1} | \mathbf{X}^k, \mathbf{X}^0) = \mathcal{N}(\mathbf{X}^{k-1}; \tilde{\mu}_k(\mathbf{X}^k, \mathbf{X}^0), \tilde{\beta}_k \mathbf{I}) \quad (5)$$

where

$$\tilde{\mu}_k(\mathbf{X}^k, \mathbf{X}^0) = (\mathbf{X}^k - \beta_k \epsilon / \sqrt{1 - \bar{\alpha}_k}) / \sqrt{\bar{\alpha}_k}.$$

and $\tilde{\beta}_k = ((1 - \bar{\alpha}_{k-1}) / (1 - \bar{\alpha}_k)) \beta_k$. To make (4) to (5), we can take

$$\mu_\theta(\mathbf{X}^k, \mathbf{X}^0) = (\mathbf{X}^k - \beta_k \epsilon_\theta(\mathbf{X}^k, k) / \sqrt{1 - \bar{\alpha}_k}) / \sqrt{\bar{\alpha}_k},$$

at diffusion step k , where $\Sigma_\theta(\mathbf{X}^k, \mathbf{X}^0) = \beta_k$ and $\epsilon_\theta(\mathbf{X}^k, k)$ is a network to approximate the noise ϵ . It turns out that the negative log-likelihood can be bounded by the following objective:

$$\sum_{k=2}^K D_{\text{KL}}(q(\mathbf{X}^{k-1} | \mathbf{X}^k, \mathbf{X}^0) \| p_\theta(\mathbf{X}^{k-1} | \mathbf{X}^k)) - \log p_\theta(\mathbf{X}^0 | \mathbf{X}^1) + D_{\text{KL}}(q(\mathbf{X}^K | \mathbf{X}^0) \| p(\mathbf{X}^K))$$

Given the model assumption (4) to (5), the main components in the objective can be down to the following term:

$$\mathbb{E}_{\mathbf{X}^k, \epsilon} \left[\frac{\beta_k^2}{2 \Sigma_\theta(\mathbf{X}^k, \mathbf{X}^0)} \|\epsilon - \epsilon_\theta(\mathbf{X}^k, k)\|^2 \right]$$

A detailed explanation of the mathematical simplification can be found in [17]. Notes that $\mathbf{X}^k = \sqrt{\bar{\alpha}_k} \mathbf{X}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$. Once trained, we can compute the following to sample from the reverse $\mathbf{X}^{k-1} \sim p_\theta(\mathbf{X}^{k-1} | \mathbf{X}^k)$,

$$\mathbf{X}^{k-1} = \frac{1}{\sqrt{\bar{\alpha}_k}} \left(\mathbf{X}^k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(\mathbf{X}^k, k) \right) + \sqrt{\Sigma_\theta} \mathbf{z}$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $k = K, \dots, 2$ and $\mathbf{z} = \mathbf{0}$ when $k = 1$. The backward process is the sampling from the most perturbed distribution $\mathbf{X}$, which is the Gaussian noise, then reduce the noises scale and make the sampling approaching to target distribution $\mathbf{X}^0$, starting from white noise sample $\mathbf{X}^K$.

2.3. Denoising network: GNE

A high-level description of the Graph-based Noise Estimation (GNE) network is as follows:

$$\epsilon_\theta^k = \epsilon_\theta(\mathbf{X}^k, h_t, \mathbf{A}, k), \quad (6)$$

where h_t is the output of GRU, $\mathbf{A}$ is the adjacency matrix, k is diffusion step and $\mathbf{X}^k$ is noisy version of the data. The module ϵ_θ is designed to approximate the diffusion noise ϵ^k , which follows a standard Gaussian distribution $\mathcal{N}(0, \mathbf{I})$, added at diffusion step k . To achieve this, we first calculate $\mathbf{X}^k$ using the following equation:

$$\mathbb{E}_{\mathbf{X}^0, \epsilon^k, k} \left[\|\epsilon^k - \epsilon_\theta \left(\sqrt{\bar{\alpha}_k} \mathbf{X}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon^k, h_t, \mathbf{A}, k \right)\|^2 \right]$$

In summary, by utilizing this objective function, we can effectively train the network ϵ_θ to approximate ϵ^k through the procedure outlined in Algorithm 1.

Algorithm 1: Training

1: Input: h_t and $\mathbf{A}$

2: repeat:

3: Sample $\mathbf{X}^0 \sim q$, $\epsilon^k \sim \mathcal{N}(0, \mathbf{I})$, and

$k \sim \text{Uniform}(\{1, \dots, K\})$

4: Take gradient step on

$$\nabla_\theta \left\| \epsilon^k - \epsilon_\theta \left(\sqrt{\bar{\alpha}_k} \mathbf{X}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon^k, h_t, \mathbf{A}, k \right) \right\|^2$$

5: until converged

Our denoising network is illustrated in Fig. 3. Drawing inspiration from WaveNet [26] and DiffWave [27], we introduce the Graph-based Noise Estimation Network (GNE) as a key component of our diffusion model. This Network architecture consists of $m = 0, 1, \dots, M-1$ residual blocks with several components as:

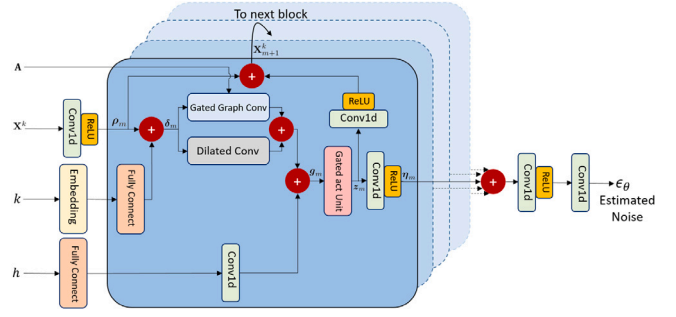


Fig. 3. The architecture of Graph-based Noise Estimation Network (GNE). In this structure, we consider both spatial features between variables and temporal features from historical data for each time series, allowing for better estimation of noise in the diffusion model.

$$\rho_m = \text{ReLU}(\text{Conv1d}(\mathbf{X}_m^k))$$

$$\delta_m = \rho_m + \text{Embed}(k) \mathbf{W}_k$$

$$g_m = \text{Conv1d}(h_t \mathbf{W}_h) + \text{dilatedConv}(\delta_m) + \text{GatedGConv}(\delta_m, \mathbf{A})$$

$$z_m = \text{GatedActivationUnit}(g_m)$$

$$\mathbf{X}_{m+1}^k = \rho_m + \text{ReLU}(\text{Conv1d}(z_m))$$

$$\eta_m = \text{ReLU}(\text{Conv1d}(z_m)),$$

where $\text{Embed}(\cdot)$ is the embedding of the diffusion step k , $\text{GatedActivationUnit}(\cdot) = \sigma(\cdot) \odot \tanh(\cdot)$ is from [28], and GatedGConv is gated graph convolution from [29]. Each residual layer contains different components for improved feature extraction. We apply Conv1d and ReLU to $\mathbf{X}^k$ for each variable in the time dimension to extract temporal features (ρ_m). To incorporate the diffusion step k into the architecture, we use embedding to project k into a high-dimensional space, followed by a fully connected layer to match the dimensions. δ_m is the summation of ρ_m and the projected version of the diffusion step. We utilize parallel feature extraction in both the temporal and spatial domains using dilated convolutions and gated graph convolutions, respectively. A dilated convolution is a convolution where the filter is applied over a larger area by skipping input values with a certain step. It is equivalent to a convolution with a larger filter derived from the original filter by dilating it with zeros, but it is significantly more efficient. A dilated convolution effectively allows the network to operate on a coarser scale than with a normal convolution, similar to pooling or strided convolutions [26]. Gated Graph Convolutions (GatedGConv) effectively capture spatial dependencies in graph-structured data [29]. We also apply a fully connected layer and Conv1d on the hidden states h_t . The summation of these features is g_m as shown in Fig. 3. We then apply a Gated Activation Unit layer, which provides a higher level of non-linearity based on [26]. Additional convolutions are applied to enhance the representation, ensuring a proper architecture for better noise estimation. All convolutions in residual blocks and weights such as $\mathbf{W}_k$ and $\mathbf{W}_h$ are learnable and included in θ . As illustrated in Fig. 3, the network eventually makes the following estimation:

$$\epsilon_\theta = \text{Conv1d}(\text{ReLU}(\text{Conv1d}(\sum_{m=0}^{M-1} \eta_m))).$$

After the training phase, we proceed to predict values for future steps of all variables. During this stage, we utilize the sampling procedure outlined in procedure outlined in Algorithm 2 to generate predictions.

Table 1
Summary statistics of datasets.

Dataset	Number of variables	Duration	Sampling interval	Number of Samples for each variable
Renewable energy	9	2019/01/01 00:00:00 — 2021/12/31 23:00:00	1 h	26 304
Electricity demand	370	2014/01/01 00:00:00 — 2014/09/08 00:00:00	1 h	6088

Algorithm 2: Sampling

```

1: Input:  $h_t$  and  $\mathbf{A}$ 
2: Sample  $\mathbf{X}^K \sim \mathcal{N}(0, \mathbf{I})$ 
3: for  $k = K, K - 1, \dots, 1$  do
4:   if  $k > 1$  then
5:      $\mathbf{Z} \sim \mathcal{N}(0, \mathbf{I})$ 
6:   else
7:      $\mathbf{Z} = 0$ 
8:   end if
 $\mathbf{X}^{k-1} = \frac{1}{\sqrt{\alpha_k}} \left( \mathbf{X}^k - \frac{\beta_k}{\sqrt{1 - \alpha_k}} \epsilon_\theta(\mathbf{X}^k, h_t, \mathbf{A}, k) \right) + \sqrt{\Sigma_\theta} \mathbf{Z}$ 
9: end for
10: Return:  $\mathbf{X}^0$ 

```

3. Use cases

We evaluated the accuracy of our proposed G-DDPM approach in forecasting applications using two real-world datasets. The essential information of these datasets is summarized in Table 1.

3.1. Use case 1: Renewable energy

This dataset was derived from the California Independent System Operator (CAISO) and the National Renewable Energy Laboratory (NREL). We use this data in 1-h intervals, underwent a rigorous cleaning and modification process to create a comprehensive database comprising three crucial time series: solar energy production, wind energy production, and electricity demand. The resulting database encompasses 10 columns, providing a rich set of variables for analysis. These include: Date: Timestamps for each data point. DHI (W/m^2): Diffuse Horizontal Irradiance, a measure of the amount of solar radiation on a horizontal surface. DNI (W/m^2): Direct Normal Irradiance, a measure of the amount of solar radiation received directly from the sun on a surface perpendicular to the sun's rays. GHI (W/m^2): Global Horizontal Irradiance, a measure of the total amount of solar radiation received on a horizontal surface, including both direct and diffuse components. Wind Speed (m/s): The speed of wind measured in meters per second. Humidity (%): The relative humidity expressed as a percentage. Temperature ($^\circ\text{C}$): The ambient temperature measured in degrees Celsius. Solar Energy Production (MW): The amount of electricity generated from solar energy sources, measured in megawatts. Wind Energy Production (MW): The amount of electricity generated from wind energy sources, measured in megawatts. Electricity Demand (MW): The total electricity demand or consumption, measured in megawatts. The data is publicly available [30].

3.2. Use case 2: Electricity demand

This benchmark dataset is extensively utilized for energy time series prediction tasks. It contains hourly electricity consumption readings from 370 distinct clients. For our analysis, we use the data collected in 2014. The dataset is structured such that each column represents the electricity consumption of an individual client, with rows corresponding to hourly time steps.

This granular temporal resolution allows for detailed analysis and modeling of electricity usage patterns. Notably, the dataset is publicly available online [31], making it a valuable resource for researchers and practitioners in the field of energy forecasting and load prediction.

4. Results & discussions

In this section, we present the experimental settings, including the data splitting strategy for training, validation, and testing. We introduce the state-of-the-art baseline methods used for validating the proposed G-DDPM method, employing various error metrics. Furthermore, an ablation study is conducted to showcase the performance of different components of the G-DDPM method. Additionally, we compare the computational time of the methods and provide an extensive discussion and analysis.

4.1. Experimental settings

The datasets were split into three subsets: a training set (60%), a validation set (20%), and a test set (20%). Independent models were trained to predict future time steps (horizons) of 24, 48, and 72, corresponding to one-day-ahead, two-day-ahead, and three-day-ahead forecasting, respectively. We performed autocorrelation analyses on all variables using the Autocorrelation Function (ACF) and Pearson's correlation coefficient across various time lags. The ACF results revealed strong autocorrelation, particularly at a 24-h lag for variables. This guided our selection of a 24-h context window for our model. The training process involved 20 epochs, 100 diffusion steps, and a linear schedule for the β parameter and to model the dependencies between variables in our simulation, we utilize the complete graph in this study. In the implementation, the dimension of the temporal data during the training phase is equal to the sum of the historical window (context length) and the prediction length (horizon). However, during the sampling phase, the temporal dimension is equal to the horizon. For complete reproducibility, the implementation of G-DDPM on these two datasets is available on GitHub.¹

4.2. Baseline methods

In this study, we compare the G-DDPM model with the following existing models:

- **DeepAR:** DeepAR is developed by Amazon for probabilistic time series forecasting. It is based on an autoregressive recurrent neural network architecture that can effectively capture complex patterns and seasonalities in time series data. The key idea behind DeepAR is to train a single model jointly over multiple related time series, leveraging the similarities between them to improve forecasting accuracy. It generates probabilistic forecasts in the form of Monte Carlo samples, allowing for the computation of quantile forecasts and uncertainty estimates [32].
- **SpecSTG:** SpecSTG is a novel spectral diffusion framework proposed for fast probabilistic spatio-temporal graph (STG) forecasting. The key idea behind SpecSTG is to generate the Fourier representation of future time series, transforming the learning process into the spectral domain enriched with spatial information. It incorporates a fast spectral graph convolution designed for Fourier input, alleviating the computational burden associated with existing models. SpecSTG adopts an autoregressive architecture, where the Fourier representation of future time series is generated recurrently conditioned on the past observations and graph structure [25].

¹ Made available at publication time.

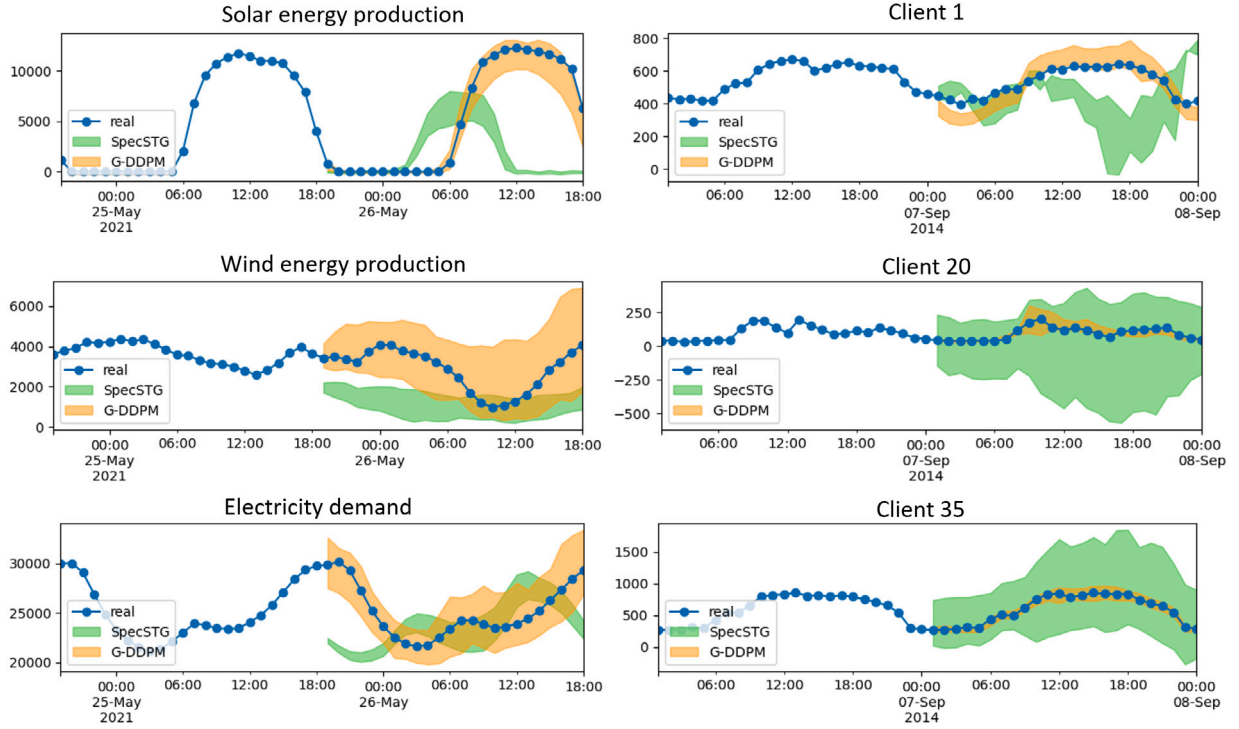


Fig. 4. Comparison of day-ahead probabilistic forecasting performance between the proposed G-DDPM and the SpecSTG method for two use cases. The right column displays variables from use case 1, while the left column displays variables from use case 2.

- GCRDD: Graph Convolutional Recurrent Denoising Diffusion Model is a diffusion-based method proposed for multivariate probabilistic temporal forecasting, particularly in spatio-temporal graph settings. It adopts an autoregressive architecture where the future time series is generated recurrently conditioned on past observations and the graph structure. GCRDD approximates the target distribution with a probabilistic density function conditional on hidden states from a graph-modified GRU and the graph topology [24].
- TimeGrad: TimeGrad is a score-based generative approach for probabilistic time series forecasting. It trains a neural network to estimate the gradient of the data distribution's log-density conditioned on past observations. This score network is then used in an annealed Langevin sampling process to generate new synthetic time series samples consistent with the learned distribution. By capturing complex temporal dependencies and multivariate correlations, TimeGrad can produce high-quality probabilistic forecasts with uncertainty estimates, making it suitable for applications like energy demand [23].
- Chronos: The Chronos framework, introduced by Amazon, presents a novel approach to time series forecasting by leveraging the capabilities of pretrained language models. The methodology involves tokenizing time series data through scaling and quantization into a fixed vocabulary, enabling the application of transformer-based architectures, such as T5, to this domain. With Chronos, we use zero-shot learning for time series forecasting, so there is no training required. We simply provide the historical data, and the framework generates predictions as output without any prior training on the specific time series [33].
- Lag-Llama: Lag-Llama is a foundation LLM-based model for probabilistic time series forecasting. It uses lagged values as covariates to capture temporal dependencies in time series data. The model is pretrained on a diverse corpus of time series data from various domains, enabling strong zero-shot generalization capabilities. Lag-Llama processes input sequences by tokenizing them along with covariates into a sequence of tokens, which are then

mapped to a hidden dimension suitable for the attention module. The model incorporates pre-normalization techniques like RM-SNorm and Rotary Positional Encoding to enhance its attention mechanism [34].

Within the baseline methods, GCRDD and SpecSTG demonstrate the strongest alignment with our proposed GDDPM, as all three address the management of dependencies between multivariate variables. However, there are key differences, particularly in the overall process and the denoising network architecture. The first distinction is in how temporal hidden states are extracted: GCRDD uses a Graph GRU (GGRU), SpecSTG employs a Spectral Graph GRU (SG-GRU), while our GDDPM method utilizes a standard GRU. The second, more critical difference lies in the denoising network. GCRDD uses four graph convolutions with Chebyshev approximation in each residual layer to capture dependencies, and SpecSTG adopts a fully spectral-domain architecture. In contrast, our GDDPM model introduces a novel approach that incorporates parallel feature extraction in each residual block, using dilated convolutions for temporal features and gated graph convolutions for spatial (graph-based) features. This design leads to richer feature representations, improving noise estimation and overall model performance.

4.3. Evaluation metrics

To evaluate the effectiveness of the proposed method, we use the following standard metrics to quantify the dissimilarity between the actual value x_t and the predicted value $\hat{x}_t$.

- Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{x}_i| \quad (7)$$

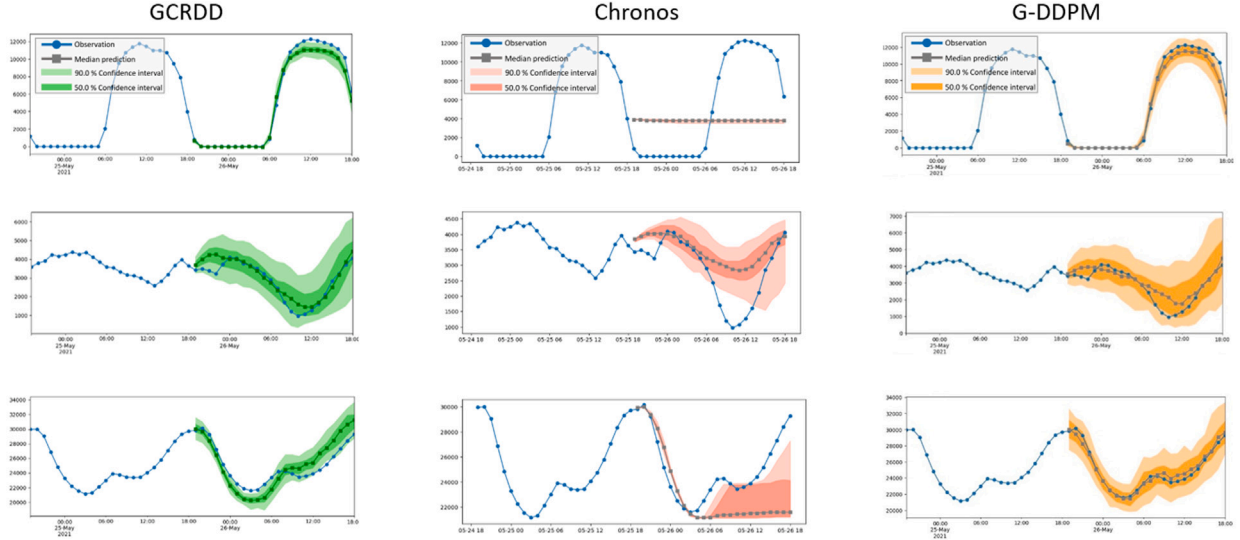
- Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2} \quad (8)$$

Table 2

Forecasting performance comparison of G-DDPM and other baseline models for Use Case 1 (Renewable Energy).

	1 day			2 days			3 days		
	MAE	RMSE	CRPS	MAE	RMSE	CRPS	MAE	RMSE	CRPS
DeepAR [32]	1183.04	1517.42	0.1811	1213.17	1554.89	0.0998	1143.99	1436.04	0.1049
SpecSTG [25]	726.18	1279.07	0.1502	698.24	1247.18	0.1652	422.80	939.80	0.0864
GCRDD [24]	435.81	938.31	0.1114	333.19	817.12	0.0870	260.50	663.44	0.0637
Timegrad [23]	411.44	878.17	0.1019	347.83	850.77	0.0818	233.32	574.95	0.0550
Chronos [33]	626.63	871.67	0.2625	510.20	755.82	0.4380	648.10	907.43	0.8177
Lag-Llama [34]	857.44	1103.01	0.0736	785.69	1030.32	0.0662	789.63	1027.64	0.0647
G-DDPM	196.348	405.77	0.0214	264.75	659.72	0.0648	311.54	748.53	0.0702
Improvement %	52.2%	53.4%	70.9%	20.5%	12.7%	2.1%	–	–	–

**Fig. 5.** Visualizations of day-ahead probabilistic forecasting results for the GCRDD, Chronos, and G-DDPM methods in Use Case 1 are shown. The first row displays solar energy production (MV), the second row represents wind energy production (MV), and the third row illustrates electricity demand (MV).

- Continuous Ranked Probability Score (CRPS) : This metric measures the compatibility of a cumulative distribution function F with an observation x as [35]:

$$\text{CRPS}(F, x) = \int_{\mathbb{R}} (F(z) - \mathbb{I}\{x \leq z\})^2 dz,$$

where $\mathbb{I}\{x \leq z\}$ is the indicator function, which equals 1 if $x \leq z$ and 0 otherwise. The CRPS is a *proper scoring function*, meaning that it achieves its minimum value when the predictive distribution F matches the true data distribution. To approximate the CRPS in practice, we can use the empirical cumulative distribution function (CDF) of F . The empirical CDF $\hat{F}(z)$ is defined as: $\hat{F}(z) = \frac{1}{K} \sum_{k=1}^K \mathbb{I}\{x^{0,k} \leq z\}$, where $x^{0,k} \sim F$ are K samples drawn from the distribution F . This empirical CDF serves as a natural approximation of the predictive CDF. By employing this empirical CDF, the CRPS can be computed directly from the simulated samples of the conditional distribution at each time point. For probabilistic time series forecasting, the CRPS metric is particularly useful as it provides a comprehensive evaluation of how well the predicted distribution captures the uncertainty and variability in the observed time series data [36].

It is worth noting that we skipped the Mean Absolute Percentage Error (MAPE) metric in this study because its values can be disproportionately high in certain cases, particularly due to the presence of values close to zero in the dataset. For instance, during nighttime hours or periods of low solar energy production, MAPE can produce misleadingly large errors. This issue arises specifically in scenarios where the data contains zero or near-zero values. To avoid these distortions and ensure more reliable performance comparisons, we opted not to use MAPE as an evaluation metric.

4.4. Results

To evaluate the performance of the proposed G-DDPM method, we conducted probabilistic forecasting experiments using data from two use cases: renewable energy and electricity demand forecasting. The results were compared with several state-of-the-art methods, including DeepAR [32], SpecSTG [25], GCRDD [24], TimeGrad [23], and Chronos [33]. Table 2 presents the prediction results for renewable energy over three different time horizons, while Table 3 shows the prediction results for electricity demand forecasting. The experimental results demonstrate the effectiveness of the G-DDPM method in accurately forecasting both use cases compared to other state-of-the-art approaches.

Tables 2 and 3 illustrate the significant performance enhancements achieved by the proposed G-DDPM model in forecasting renewable energy and electricity demand compared to various baseline models. The most notable improvements are observed over other advanced methods, particularly in the case of Timegrad and DeepAR. For Use Case 2 (Electricity Demand), G-DDPM outperforms Timegrad with an 8.04% lower MAE and a 24.30% lower RMSE for 1-day ahead forecasts. Similarly, G-DDPM demonstrates an 85.44% improvement in MAE and a 15.34% improvement in RMSE over DeepAR for the same 1-day forecasting horizon. These substantial gains highlight G-DDPM's advanced capability to capture complex temporal dependencies and refine predictions effectively.

In Use Case 1 (Renewable Energy), G-DDPM continues to deliver impressive results, particularly when compared to models like SpecSTG and GCRDD. For 1-day ahead predictions, G-DDPM achieves a 72.97% reduction in MAE and a 68.29% reduction in RMSE over SpecSTG. Over a 2-day forecast, the model shows a significant 20.53% improvement in

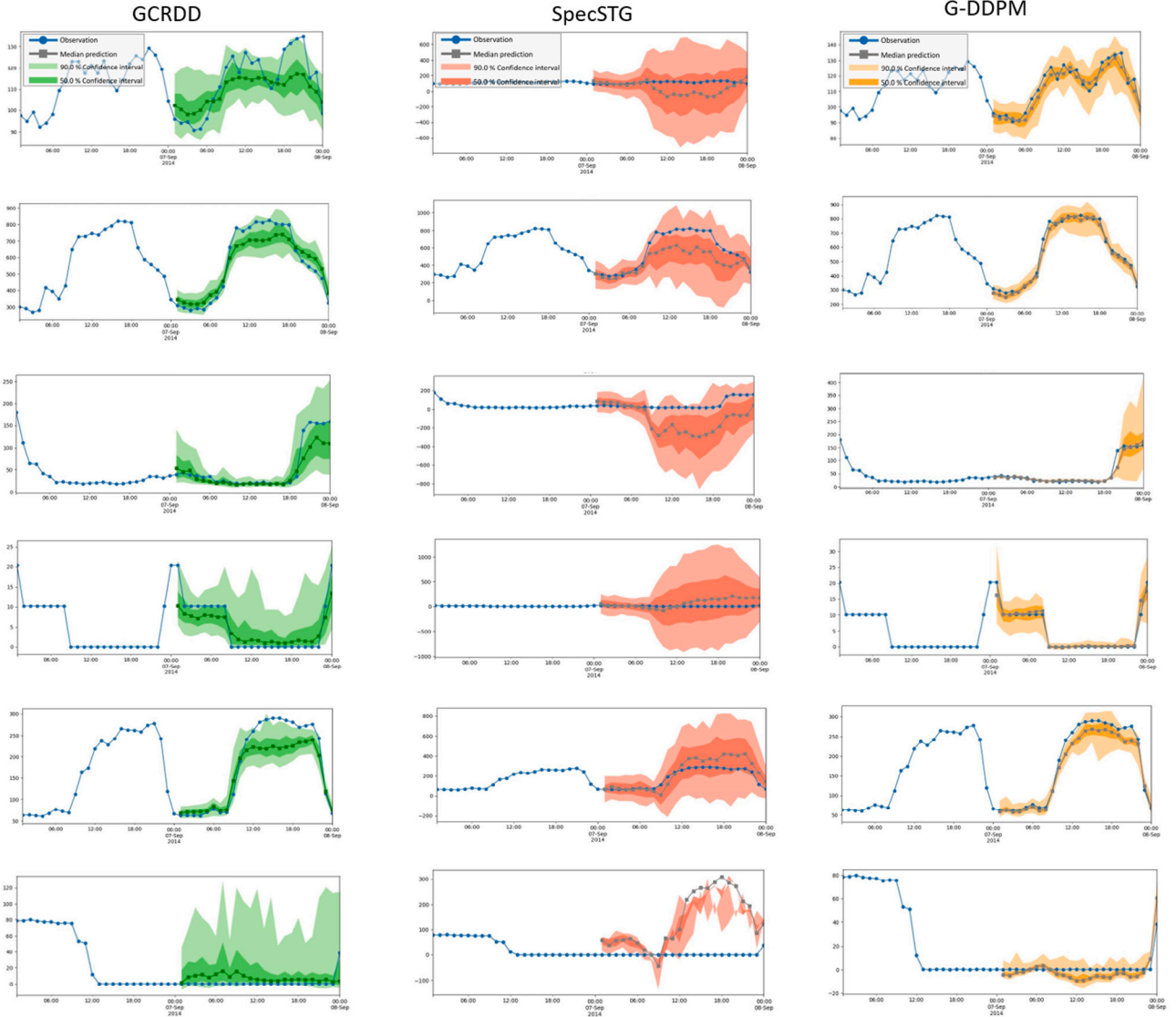


Fig. 6. Visualizations of day-ahead probabilistic forecasting results for the GCRDD, SpecSTG, and G-DDPM methods are presented for use case 2. This showcases six different clients, each with a distinct demand pattern, where the demand is measured in kilowatts (kW).

MAE and a 19.23% improvement in RMSE compared to GCRDD. These improvements demonstrate G-DDPM's robust performance in various forecasting scenarios.

The G-DDPM model's superior performance extends across both use cases, consistently outperforming other contemporary models. Its iterative refinement approach in denoising diffusion methods allows it to model intricate temporal relationships more effectively than traditional models. Conversely, Chronos, which uses zero-shot learning for time-series forecasting, exhibits notably poor performance. It struggles with modeling specific temporal patterns, resulting in significantly higher error metrics. For example, Chronos shows a 262.5% worse result in CRPS for 1-day predictions compared to G-DDPM. It is worth noting that the Chronos model was trained on benchmark dataset 2, and the test portion of the data was seen by this model during training. Consequently, it would not be a fair comparison to include the results of the Chronos model in Table 3. Therefore, we have omitted the Chronos model's results from Table 3 to ensure a fair and unbiased evaluation. Overall, G-DDPM's ability to handle complex temporal dependencies and provide significant improvements in predictive accuracy makes it a standout choice for probabilistic time-series forecasting.

Figs. 4, 5, and 6 visually demonstrate the superior performance of G-DDPM compared to other methods. Specifically, in Fig. 4, we compare

G-DDPM with the recent method SpecSTG, both of which are diffusion-based and consider graph modeling between variables. For use case 1, we provide predictions for solar energy production, wind energy production, and electricity demand. In Fig. 4 (use case 2), we display results for clients numbered 1, 20, and 35. As the results indicate, G-DDPM outperforms other methods significantly. Particularly for certain variables, such as wind energy production, there is expected high uncertainty, which is effectively captured in the results by G-DDPM.

Fig. 5 compares the day-ahead forecasting results of the proposed G-DDPM method with the GCRDD and Chronos approaches for Use Case 1. The results demonstrate that pre-trained language models like Chronos yield unacceptable performance. Between GCRDD and G-DDPM, both of which consider variable dependencies in the diffusion model, G-DDPM exhibits superior performance. For instance, in forecasting electricity demand, our method more closely tracks the actual observed values. Fig. 6 visually compares the day-ahead forecasting results of the proposed G-DDPM method with GCRDD and SpecSTG for Use Case 2. We selected clients with different demand patterns, and the results indicate that SpecSTG does not perform well, although we will demonstrate in the next section that this method is computationally fast. Additionally, we can clearly observe the superior performance of G-DDPM compared to GCRDD.

Table 3

Forecasting performance comparison of G-DDPM and other baseline models for Use Case 2 (Electricity Demand).

	1 day			2 days			3 days		
	MAE	RMSE	CRPS	MAE	RMSE	CRPS	MAE	RMSE	CRPS
DeepAR [32]	319.83	410.01	0.0544	326.02	417.30	0.0549	334.81	424.56	0.0526
SpecSTG [25]	201.03	500.76	0.0679	313.66	863.56	0.1157	912.62	3245.22	0.4117
GCRDD [24]	65.39	621.47	0.0651	71.56	637.54	0.0712	66.24	557.89	0.0669
Timegrad [23]	50.64	458.57	0.0521	51.99	464.22	0.0521	53.51	413.26	0.0531
Lag-Llama [34]	325.53	415.99	0.0613	334.64	424.03	0.0718	353.54	457.18	0.0759
G-DDPM	46.56	347.13	0.0491	49.66	383.94	0.0502	50.06	355.33	0.0499
Improvement %	8%	15.3%	5.7%	4.4%	7.9%	3.6%	6.4%	14%	5.1%

Table 4

Ablation study.

Method	G-DDPM	w/o GGC	w/o DC	w/o GGC & DC
MAE	46.56	50.64	51.49	272.95
RMSE	347.13	458.57	398.03	2652.53
CRPS	0.0491	0.0521	0.0523	0.2829

4.5. Ablation study

To further validate the effectiveness of our proposed G-DDPM framework, we conducted an ablation study focusing on the GNE block. We examined the impact of each primary component within the GNE by systematically removing them and analyzing the resulting changes in performance. This analysis led to the following configurations:

- **w/o GGC:** G-DDPM without the Gated Graph Convolution (GGC). In this setup, graph modeling is excluded, meaning the model does not directly consider the interdependencies between variables. Instead, it relies on a simplified approach that does not leverage the rich relational information typically captured by the GGC.
- **w/o DC:** G-DDPM without the Dilated Convolution (DC). This configuration ignores the explicit temporal features, leading to a less effective capture of time-based patterns in the data.
- **w/o GGC & DC:** G-DDPM without both the Gated Graph Convolution and Dilated Convolution. This version removes all forms of convolution, resulting in a basic model that lacks both spatial (graph-based) and temporal (time-based) feature extraction.

Table 4 presents the forecasting errors evaluated by three metrics for one day ahead forecasting in Use Case 2.

The ablation analysis underscores the importance of each component in the G-DDPM architecture. The results in Table 4 demonstrate that the main components within the GNE block significantly contribute to the model's overall performance. Specifically, removing the GGC results in performance degradations: MAE deteriorates by approximately 8.8%, RMSE increases substantially by 32.1%, and CRPS reflects a degradation of about 6.1%. These results indicate that the GGC is crucial for precise forecasting as it effectively captures the interdependencies between variables. Removing the DC leads to an increase in MAE by 10.6%, a rise in RMSE by 14.7%, and an increase in CRPS by 6.5%, highlighting the importance of temporal features for accurately modeling time-series data. When both GGC and DC are removed, the model's performance dramatically deteriorates: MAE increases by 486.1%, RMSE rises by 663.9%, and CRPS jumps by 476.1%. These significant performance drops underscore the essential roles of both spatial (graph-based) and temporal (time-based) feature extractions for the robustness and effectiveness of the G-DDPM model. This ablation study clearly illustrates that both the Gated Graph Convolution and Dilated Convolution are indispensable components of the G-DDPM

Table 5

Computational running times.

	Training time (s)		Inference time (s)	
	Use case 1	Use case 2	Use case 1	Use case 2
DeepAR [32]	9	740	0.05	1.97
SpecSTG [25]	23	37	0.59	0.62
GCRDD [24]	22	22	30	30
Timegrad [23]	7	28	0.58	0.62
Chronos [33]	–	–	0.18	7.5
Lag-Llama [34]	–	–	1.19	0.95
G-DDPM	9	47	1.08	1.58

framework. Their presence significantly enhances the model's ability to forecast accurately, demonstrating their crucial roles in capturing the complex dynamics and interdependencies inherent in the data.

4.6. Computation time

We evaluated the computational cost of the methods discussed in this paper, and the results are presented in Table 5. The comparison of computational running times across different forecasting methods reveals interesting insights into their strengths and trade-offs during training and inference. All model training and testing were conducted on Google Colab, utilizing its freely available cloud-based resources. The typical hardware configuration included an NVIDIA Tesla T4 GPU with 16 GB of VRAM, an Intel Xeon processor (ranging from 2 to 4 cores), and between 12 GB and 16 GB of RAM. DeepAR [32] displays a notable variability in training times depending on the use case, indicating potential challenges in scaling up to more intricate scenarios. SpecSTG [25] impresses with its consistent and relatively swift training times, showcasing adaptability and efficiency. GCRDD [24] achieves robust performance coupled with stable training times, although it falls short in terms of inference efficiency due to significantly higher times, which may deter its use in real-time applications. Timegrad [23] and SpecSTG [25] share a common advantage in their efficient training and inference capabilities, making them well-suited for environments demanding rapid responses. Chronos [33] is noteworthy for its exceptionally low inference times in simpler cases, but this advantage seems to wane in more complex situations, where its inference times lengthen considerably. Our proposed method, G-DDPM, emerges as a well-balanced contender, offering competitive training times and consistent inference performance. While it might not be the fastest, it delivers reliable efficiency and excels in prediction accuracy, making it a compelling choice for applications where both computational efficiency and robustness are essential requirements.

4.7. Practical consideration and future work

While our proposed denoising diffusion method for multivariate probabilistic time series forecasting has achieved state-of-the-art performance in renewable energy and electricity demand forecasting, there

Table 6

Table of hyperparameters.

Model	Diffusion steps	Input length	β schedule	β end	Cell type	Batches per Epoch	Loss type	Number of Epoch
G-DDPM	100	24	Linear	0.1	GRU	100	L2	20
SpecSTG [25]	100	24	Linear	0.1	Graph GRU	100	L2	20
Timegrad [23]	100	24	Linear	0.1	GRU	100	L2	20
GCRDD [24]	100	24	Linear	0.1	GRU	50	L2	20
DeepAR [32]	–	24	–	–	LSTM	50	Quantile loss	20
Chronos [33]	–	24	–	–	–	–	–	–
Lag-Llama [34]	–	24	–	–	–	–	–	–

are several practical considerations and research directions that can further enhance its impact and applicability. Ensuring scalability and computational efficiency can be addressed by leveraging more efficient algorithms and distributed computing architectures. The quality and availability of data are critical, so implementing robust techniques to handle missing data and incorporate diverse sources will improve model robustness. In this study, while we prioritized optimizing key parameters, such as the diffusion step, a full grid search across all parameters was not conducted due to computational limitations. To achieve optimal performance, we recommend a more exhaustive parameter tuning process prior to deploying our method. Enhancing interpretability through methods like feature attribution and saliency maps is also essential. In our future research, we aim to prioritize addressing the question of which type of graph offers better performance and explainability for datasets without an associated physical graph. Additionally, we will focus on integrating the method with renewable energy sources and developing real-time forecasting capabilities and decision support systems. These advancements are crucial for enhancing our method's utility in delivering reliable forecasts and facilitating effective energy management during the transition towards sustainable energy systems.

5. Conclusion

This paper proposes a novel denoising diffusion method for multivariate probabilistic time series forecasting tailored to renewable energy production and electricity demand. By explicitly modeling variable interdependencies through graph modeling and capturing temporal dynamics, our approach achieves state-of-the-art performance on two real-world datasets. Extensive experiments demonstrate reductions of 2.1%–70.9% in Continuous Ranked Probability Score (CRPS) and 7.9%–53.4% in Root Mean Squared Error (RMSE) compared to existing methods. Our framework provides reliable probabilistic forecasts crucial for effective decision-making and resource allocation in the energy sector, addressing uncertainties associated with intermittent renewables and fluctuating demand. The explicit modeling of variable interdependencies and temporal information enables accurate probabilistic forecasting for modern power systems and facilitates the transition towards sustainable energy sources.

CRediT authorship contribution statement

Amir Miraki: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation. **Pekka Parviainen:** Writing – review & editing, Supervision, Methodology, Investigation. **Reza Arghandeh:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

A.1. Implementation details

In this section, we provide the details of our implementation for reproducibility. The hyperparameters used in our experiments are as follows: for the G-DDPM method, the architecture is configured with a “number of Graph Gated layers” of 1, a “number of cells” of 40, an “embedding dimension” of 5, a “number of residual layers” of 8, a “dilation cycle” of 2, and a “Condition Length” of 100; for the TimeGrade method, the model is parameterized with a “number of cells” of 40, an “embedding dimension” of 5, a “number of residual layers” of 8, a “dilation cycle” of 2, and a “Condition Length” of 100; for the SpecSTG method, the “SpecConv polynomial level” is set to 2 and the “number of cells” is 64; for the GCRDD method, the “ChebConv polynomial level” is set to 2; and for the DeepAR method, the architecture consists of a “Number of Layers” of 2, a “number of cells” of 40, and we utilize the pre-trained Chronos model “chronos-t5” with 8M parameters and a “number of samples” of 100. Lag-Llama has around 2.5 million parameters, and the number of samples is 100. Additional hyperparameters are presented in Table 6.

A.2. Details of GRU

A Gated Recurrent Unit (GRU) is a type of recurrent neural network architecture designed to address the vanishing gradient problem in traditional RNNs. It uses two gates - an update gate and a reset gate - to control the flow of information through the network. The update gate determines how much of the previous hidden state should be retained, while the reset gate decides how much of the previous hidden state should be forgotten when computing the new candidate hidden state. The key equations for a GRU are:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t] + b)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

where z_t is the update gate, r_t is the reset gate, $\tilde{h}_t$ is the candidate hidden state, and h_t is the final hidden state at time step t . The GRU's simpler structure compared to LSTMs makes it computationally more efficient while still maintaining the ability to capture long-term dependencies in sequential data. In our model, GRU serves as a crucial component for capturing temporal hidden features. The GRU's ability to learn long-term dependencies in sequential data makes it particularly well-suited for this task. To validate our choice of using GRU, we conducted comparative experiments with alternative layers. Table 7 presents these additional results, demonstrating the performance of our method when replacing the GRU with other types of layers. These comparisons highlight the effectiveness of the GRU in our specific application and provide insights into the model's sensitivity to different recurrent architectures.

The results demonstrate that the use of GRU in the G-DDPM model significantly improves performance. The G-DDPM with GRU outperforms models using Fully Connected (FC) layers, Long Short-Term

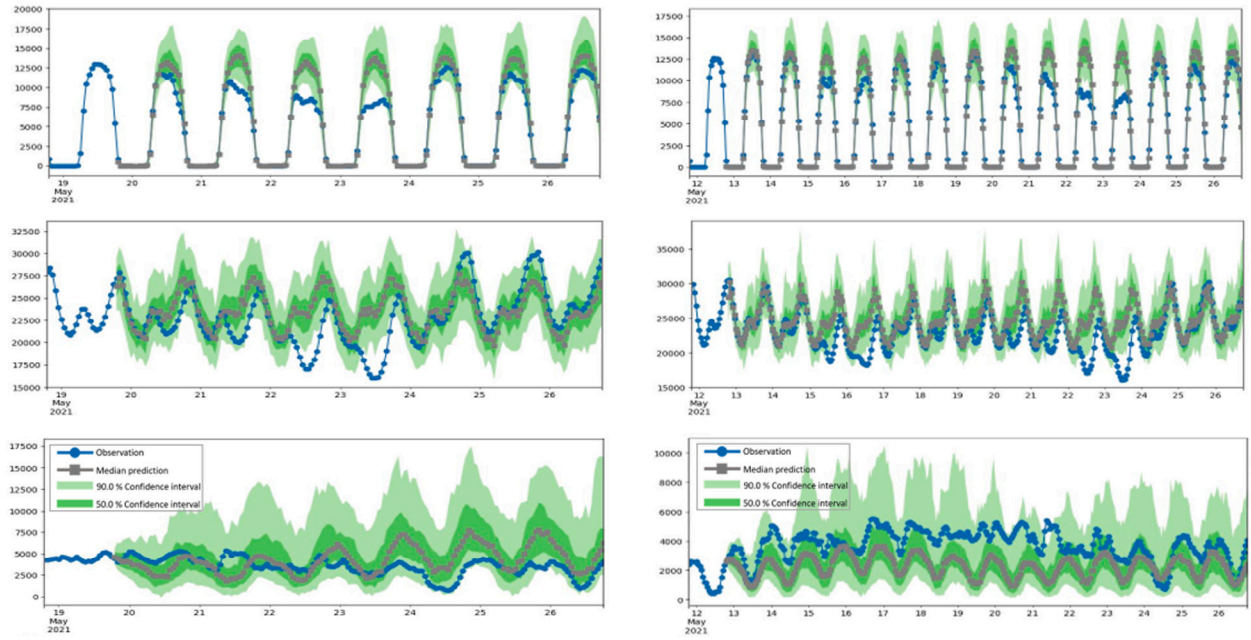


Fig. 7. Visualizations of long-term probabilistic forecasting results for the G-DDPM method in Use Case 1 are shown. The first row displays solar energy production (MV), the second row represents wind energy production (MV), and the third row illustrates electricity demand (MV). The left column is for one week ahead forecasting and the right column is for two weeks ahead forecasting results.

Table 7
Performance analysis of model using different recurrent layer architectures.

Method	G-DDPM	w/o GRU	+ FC	+ LSTM	+ Transformer
MAE	46.56	66.83	47.57	48.08	83.34
RMSE	347.13	555.14	356.43	370.90	845.37
CRPS	0.0491	0.0670	0.0583	0.0492	0.0853

Memory (LSTM) units, and Transformer blocks. Interestingly, the model incorporating Transformer blocks shows the poorest performance among all tested architectures. This is a surprising outcome, given the general success of Transformers in various sequence modeling tasks.

A.3. Analyzing forecast horizons and uncertainty

In this section, we conducted an analysis to examine the increase in uncertainty as the forecast horizon extends. Generally, it is logical to expect that as the prediction horizon expands, the level of uncertainty gradually increases. However, the methods investigated in this study appear to deviate from this hypothesis. To facilitate a more comprehensive understanding, Fig. 7 illustrates the prediction results of the G-DDPM model for Use Case 1, comparing forecasts for both one-week and two-week horizons. The results demonstrate that the forecasting has been effectively performed for all three variables. Notably, the degree of uncertainty does not exhibit a significant or discernible increase. This characteristic may be attributed to the cyclical nature of the data and its consistent repetition. Nevertheless, this feature could potentially be considered a limitation of the models examined in this paper. The lack of increased uncertainty over longer prediction horizons, while potentially beneficial in some contexts, may not accurately reflect the inherent unpredictability of long-term forecasts in certain domains. This observation warrants further investigation and may provide avenues for future research in improving the uncertainty quantification of forecasting models, particularly for extended time horizons.

Data availability

The datasets used in this paper are publicly available.

References

- [1] Ahmed Adil, Khalid Muhammad. A review on the selected applications of forecasting models in renewable power systems. *Renew Sustain Energy Rev* 2019;100:9–21.
- [2] Panwar NL, Kaushik SC, Kothari Surendra. Role of renewable energy sources in environmental protection: A review. *Renew Sustain Energy Rev* 2011;15(3):1513–24.
- [3] Wang Huaizhi, Lei Zhenxing, Zhang Xian, Zhou Bin, Peng Jianchun. A review of deep learning for renewable energy forecasting. *Energy Convers Manage* 2019;198:111799.
- [4] Vu Dao Hoang, Muttaqi Kashem M, Agalgaonkar Ashish P, Bouzerdoum Abdesselam. Short-term electricity demand forecasting using autoregressive based time varying model incorporating representative data adjustment. *Appl Energy* 2017;205:790–801.
- [5] Mahia Farjana, Dey Arpita Rani, Masud Md Abdul, Mahmud Mohammad Sultan. Forecasting electricity consumption using ARIMA model. In: 2019 international conference on sustainable technologies for industry 4.0. STI, IEEE; 2019, p. 1–6.
- [6] Taylor James W. Short-term electricity demand forecasting using double seasonal exponential smoothing. *J Oper Res Soc* 2003;54(8):799–805.
- [7] Setiawan Anthony, Koprinska Irena, Agelidis Vassilios G. Very short-term electricity load demand forecasting using support vector regression. In: 2009 international joint conference on neural networks. IEEE; 2009, p. 2888–94.
- [8] Kariniotakis Georges. Renewable energy forecasting: from models to applications. Woodhead Publishing; 2017.
- [9] Miraki Amir, Parviainen Pekka, Arghandeh Reza. Electricity demand forecasting at distribution and household levels using explainable causal graph neural network. *Energy AI* 2024;16:100368.
- [10] Hyndman Rob J, Fan Shu. Density forecasting for long-term peak electricity demand. *IEEE Trans Power Syst* 2009;25(2):1142–53.
- [11] Wang Jian-Zhou, Wang Yun, Jiang Ping. The study and application of a novel hybrid forecasting model—A case study of wind speed forecasting in China. *Appl Energy* 2015;143:472–88.
- [12] Jin Huaiping, Shi Lixian, Chen Xiangguang, Qian Bin, Yang Biao, Jin Huaikang. Probabilistic wind power forecasting using selective ensemble of finite mixture Gaussian process regression models. *Renew Energy* 2021;174:1–18.
- [13] Lauret Philippe, David Mathieu, Pedro Hugo TC. Probabilistic solar forecasting using quantile regression models. *Energies* 2017;10(10):1591.

- [14] Hong Tao, Pinson Pierre, Fan Shu, Zareipour Hamidreza, Troccoli Alberto, Hyndman Rob J. Probabilistic energy forecasting: Global energy forecasting competition 2014 and beyond. 2016.
- [15] Simeunović Jelena, Schubnel Baptiste, Alet Pierre-Jean, Carrillo Rafael E. Spatio-temporal graph neural networks for multi-site PV power forecasting. *IEEE Trans Sustain Energy* 2021;13(2):1210–20.
- [16] Kaur Devinder, Islam Shama Naz, Mahmud Md Apel, Haque Md Enamul, Dong Zhao Yang. Energy forecasting in smart grid systems: recent advancements in probabilistic deep learning. *IET Gener Transm Distrib* 2022;16(22):4461–79.
- [17] Ho Jonathan, Jain Ajay, Abbeel Pieter. Denoising diffusion probabilistic models. *Adv Neural Inf Process Syst* 2020;33:6840–51.
- [18] Dhariwal Prafulla, Nichol Alexander. Diffusion models beat gans on image synthesis. *Adv Neural Inf Process Syst* 2021;34:8780–94.
- [19] Popov Vadim, Vovk Ivan, Gogoryan Vladimir, Sadekova Tasnima, Kudinov Mikhail. Grad-tts: A diffusion probabilistic model for text-to-speech. In: *International conference on machine learning*. PMLR; 2021, p. 8599–608.
- [20] Cao Hanqun, Tan Cheng, Gao Zhangyang, Xu Yilun, Chen Guangyong, Heng Pheng-Ann, Li Stan Z. A survey on generative diffusion models. *IEEE Trans Knowl Data Eng* 2024.
- [21] Yang Ling, Zhang Zhilong, Song Yang, Hong Shenda, Xu Runsheng, Zhao Yue, Zhang Wentao, Cui Bin, Yang Ming-Hsuan. Diffusion models: A comprehensive survey of methods and applications. *ACM Comput Surv* 2023;56(4):1–39.
- [22] Capel Esteban Hernandez, Dumas Jonathan. Denoising diffusion probabilistic models for probabilistic energy forecasting. In: *2023 IEEE belgrade powerTech*. IEEE; 2023, p. 1–6.
- [23] Rasul Kashif, Seward Calvin, Schuster Ingmar, Vollgraf Roland. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In: *International conference on machine learning*. PMLR; 2021, p. 8857–68.
- [24] Li Ruikun, Li Xuliang, Gao Shiyang, Choy ST Boris, Gao Junbin. Graph convolution recurrent denoising diffusion model for multivariate probabilistic temporal forecasting. In: *International conference on advanced data mining and applications*. Springer; 2023, p. 661–76.
- [25] Lin Lequan, Shi Dai, Han Andi, Gao Junbin. SpecSTG: A fast spectral diffusion framework for probabilistic spatio-temporal traffic forecasting. 2024, arXiv preprint [arXiv:2401.08119](https://arxiv.org/abs/2401.08119).
- [26] Oord Aaron van den, Dieleman Sander, Zen Heiga, Simonyan Karen, Vinyals Oriol, Graves Alex, Kalchbrenner Nal, Senior Andrew, Kavukcuoglu Koray. Wavenet: A generative model for raw audio. 2016, arXiv preprint [arXiv:1609.03499](https://arxiv.org/abs/1609.03499).
- [27] Kong Zhifeng, Ping Wei, Huang Jiaji, Zhao Kexin, Catanzaro Bryan. Diffwave: A versatile diffusion model for audio synthesis. 2020, arXiv preprint [arXiv:2009.09761](https://arxiv.org/abs/2009.09761).
- [28] Van den Oord Aaron, Kalchbrenner Nal, Espeholt Lasse, Vinyals Oriol, Graves Alex, et al. Conditional image generation with pixelcnn decoders. *Adv Neural Inf Process Syst* 2016;29.
- [29] Li Yujia, Tarlow Daniel, Brockschmidt Marc, Zemel Richard. Gated graph sequence neural networks. 2015, arXiv preprint [arXiv:1511.05493](https://arxiv.org/abs/1511.05493).
- [30] Ortega Rojas, et al. Renewable energy and electricity demand time series dataset. 2023, <http://dx.doi.org/10.17632/fdftr3tc2.1>, UMDendley Data.
- [31] Trindade Artur. ElectricityLoadDiagrams20112014. 2015, <http://dx.doi.org/10.24432/C58C86>, UCI Machine Learning Repository.
- [32] Salinas David, Flunkert Valentin, Gasthaus Jan, Januschowski Tim. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *Int J Forecast* 2020;36(3):1181–91.
- [33] Ansari Abdul Fatir, Stella Lorenzo, Turkmen Caner, Zhang Xiyuan, Mercado Pedro, Shen Huibin, Shchur Oleksandr, Rangapuram Syama Sundar, Arango Sebastian Pineda, Kapoor Shubham, et al. Chronos: Learning the language of time series. 2024, arXiv preprint [arXiv:2403.07815](https://arxiv.org/abs/2403.07815).
- [34] Rasul Kashif, Ashok Arjun, Williams Andrew Robert, Ghonia Hena, Bhagwatkar Rishika, Khorasani Arian, Bayazi Mohammad Javad Darvishi, Adamopoulos George, Riachi Roland, Hassen Nadhir, et al. Lag-llama: Towards foundation models for probabilistic time series forecasting. 2024, Preprint.
- [35] Matheson James E, Winkler Robert L. Scoring rules for continuous probability distributions. *Manage Sci* 1976;22(10):1087–96.
- [36] Jordan Alexander, Krüger Fabian, Lerch Sebastian. Evaluating probabilistic forecasts with scoringrules. 2017, arXiv preprint [arXiv:1709.04743](https://arxiv.org/abs/1709.04743).